Cours 2

Vérification déductive

Logique de Hoare

Annotations JML : conception par contrats pour Java

Logique du premier ordre

Java et la logique dynamique

Vérification de programmes itératifs - invariants et variants de boucles

De Java/JML à JavaDL

C/ACSL et Frama-C

Vérification incrémentale

Preuve interactive

Logique de Hoare

Logique pour raisonner sur la correction partielle d'un programme impératif par rapport à une précondition et une postcondition

- Logique de programmes
- Système formel : règles d'inférence
- Autre nom : sémantique axiomatique

[Hoa69] C. A. R. Hoare.
An axiomatic basis for computer programming.
Commun. ACM, 12(10):576–580, 1969.

Mini-langage de programmes impératifs

Commandes (instructions) :

$c, c' ::=$	$\mid x := e$	affectation
	$\mid c; c'$	séquence
	$\mid$ if b then c else c' fi	condition
	$\mid$ while b do c od	répétition

- x est une variable, e est une expression (du même type que x) et b est une expression booléenne
- Langage des "tant-que" programmes de [Hoa69] + if-then-else
- "fi" et "od" ajoutés pour éviter les confusions d'interprétation des commandes
  if b then c_1 else c_2; c_3 et while b do c_1; c_2
- Langage Turing-complet
- 2 comportements : terminaison ou divergence

Triplet de Hoare

Triplet de Hoare

Un triplet de Hoare $\{P\}\ c\ \{Q\}$ est composé d'une commande c et de deux formules : la précondition P et la postcondition Q.

- P et Q sur des propriétés d'état : formules portant sur les variables de la commande c
- Le triplet de Hoare $\{P\}\ c\ \{Q\}$ est valide si toute exécution de c à partir d'un état qui satisfait la précondition P soit ne termine pas, soit termine dans un état qui satisfait la postcondition Q.
- Formulation plus courte : Le triplet de Hoare $\{P\}\ c\ \{Q\}$ est valide si, lorsque la précondition P est satisfaite, la commande c, si elle termine, établit la postcondition Q
- On parle ici de correction partielle : on ignore les cas de non-terminaison (divergence) de la commande
- La logique de Hoare est un système déductif correct et complet pour déduire les triplets de Hoare valides : une règle par sorte de commande + règle de conséquence

Logique de Hoare

Sémantique axiomatique

$$\frac{}{\{P\ (e/x)\}\ x := e\ \{P\}}\ \text{(assignment)}$$

$$\frac{\{P\}\ c\ \{Q\} \qquad \{Q\}\ c'\ \{R\}}{\{P\}\ c; c'\ \{R\}}\ \text{(sequence)}$$

$$\frac{\{b \wedge P\}\ c_1\ \{Q\} \qquad \{\neg b \wedge P\}\ c_2\ \{Q\}}{\{P\}\ \text{if } b \text{ then } c_1 \text{ else } c_2 \text{ fi}\ \{Q\}}\ \text{(if-then-else)}$$

$$\frac{\{b \wedge P\}\ c\ \{P\}}{\{P\}\ \text{while } b \text{ do } c \text{ od}\ \{\neg b \wedge P\}}\ \text{(while)}$$

$$\frac{P \Rightarrow P' \qquad \{P'\}\ c\ \{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\ c\ \{Q\}}\ \text{(consequence)}$$

Règle pour l'affectation

$$\frac{}{\{P\ (e/x)\}\ x := e\ \{P\}}\ \text{(assignment)}$$

- Axiome (pas de prémisse)
- (e/x) formalise la substitution (remplacement uniforme) de x par e
- $P\ (e/x)$ désigne la formule obtenue en remplaçant par e chaque variable x (libre) dans P
- L'affectation $x := e$ ne modifie l'état qu'en donnant la valeur de l'expression e à la variable x. Par conséquent, la formule P (portant sur x) est vraie après exécution de la commande $x := e$ si la formule $P\ (e/x)$ est vraie avant cette affectation.
- Exemples
  1. Donner une formule F telle que le triplet $\{F\}\ x := 2x + 3\ \{y = 4x + 5\}$ soit dérivable (par la règle d'affectation).

     F peut être ..... $[R_2]$

  2. Même question pour le triplet $\{F\}\ x := x + 1\ \{(\exists x.y > 4x + 3) \wedge y < 6x\}$

     F peut être ..... $[R_3]$

Règle pour la composition séquentielle

$$\frac{\{P\}\ c\ \{Q\} \qquad \{Q\}\ c'\ \{R\}}{\{P\}\ c; c'\ \{R\}}\ \text{(sequence)}$$

- Pour dériver un triplet de la forme $\{P\}\ c; c'\ \{R\}$ (i.e. démontrer qu'il est valide), il faut **deviner** une formule Q telle que les triplets $\{P\}\ c\ \{Q\}$ et $\{Q\}\ c'\ \{R\}$ soient dérivables !
- Documenter le programme avec Q ? $\rightsquigarrow$ notion d'assertion
- Calculer Q ? $\rightsquigarrow$ notion de plus faible précondition

Règle pour l'instruction conditionnelle

$$\frac{\{b \wedge P\}\ c_1\ \{Q\} \qquad \{\neg b \wedge P\}\ c_2\ \{Q\}}{\{P\}\ \text{if } b \text{ then } c_1 \text{ else } c_2 \text{ fi}\ \{Q\}}\ \text{(if-then-else)}$$

- On suppose que l'expression booléenne b est aussi une formule (portant sur les variables d'état)
- Sinon, remplacer b par la formule $b = \mathit{true}$, et $\neg b$ par $b = \mathit{false}$

Règle pour l'instruction itérative

$$\frac{\{b \wedge P\}\ c\ \{P\}}{\{P\}\ \text{while } b \text{ do } c \text{ od}\ \{\neg b \wedge P\}}\ \text{(while)}$$

- La formule P est un invariant de boucle
- La règle se lit "Si l'invariant est satisfait au début de la boucle (précondition P) et si une itération quelconque de la boucle préserve l'invariant (prémisse $\{b \wedge P\}\ c\ \{P\}$), alors (si la boucle termine) l'état après la boucle satisfait la négation de la condition de boucle et l'invariant (postcondition $\neg b \wedge P$)"
- **Après la boucle, l'invariant est encore vrai** (et la condition de boucle est fausse)
- Il faut **deviner** cet invariant P
- Calculer P ? pas général, difficile, $\rightsquigarrow$ notion de synthèse d'invariant
- En général, il faut démontrer un triplet de la forme

$$\{P\}\ \text{while } b \text{ do } c \text{ od}\ \{Q\}$$

où Q n'est pas équivalent à $\neg b \wedge P \rightsquigarrow$ règle de conséquence

Règle de conséquence

$$\frac{P \Rightarrow P' \qquad \{P'\}\ c\ \{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\ c\ \{Q\}}\ \text{(consequence)}$$

- Pour "recoller les morceaux"
- Cas typique

$$\frac{P \Rightarrow I \qquad \{I\}\ \text{while } b \text{ do } c \text{ od}\ \{\neg b \wedge I\} \qquad \neg b \wedge I \Rightarrow Q}{\{P\}\ \text{while } b \text{ do } c \text{ od}\ \{Q\}}\ \text{(consequence)}$$

Exemple de preuve en logique de Hoare (1/3)

$$\frac{\dfrac{}{\{n \geq 0\}\ i := 0\ \{n \geq 0 \wedge i = 0\}}\ (\textit{assignment}) \qquad (1)}{\{n \geq 0\}\ i := 0;\ r := 1;\ \text{while } i < n \text{ do } i := i + 1;\ r := r * i \text{ od}\ \{r = n!\}}\ (\textit{sequence})$$

où (1) est

$$\frac{\dfrac{}{\ldots\ldots\ldots

Exemple de preuve en logique de Hoare (3/3)

2. Une itération préserve l'invariant (suite et fin)

$$\dfrac{\dfrac{\{i < n \wedge r = i!\}\ i := i+1\ \{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{16}]}\},\quad \{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{17}]}\}\ r := r * i\ \{i \leq n \wedge r = i!\}}{\{i < n \wedge i \leq n \wedge r = i!\}\ i := i+1;\ r := r * i\ \{i \leq n \wedge r = i!\}}\ (seq.)}{\{i \leq n \wedge r = i!\}\ \text{while } i < n \text{ do } i := i+1;\ r := r * i \text{ od } \{i \not< n \wedge i \leq n \wedge r = i!\}}\ (while)$$

3. Négation de la condition de boucle et invariant impliquent la postcondition

$$\neg(i < n)\ \wedge \ldots\ldots\ldots\ldots\ldots\ldots^{[R_{18}]} \Rightarrow r = n!$$

car

$$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{19}]}$$

CQFD.

Annotation du code par des assertions

A priori, entre chaque instruction

$\{n \geq 0\}$
$i := 0;$
$\{a_1\}$
$r := 1;$
$\{a_2\}$
while $i < n$ do
  $\{a_3\}$
  $i := i + 1;$
  $\{a_4\}$
  $r := r * i$
  $\{a_5\}$
od
$\{a_6\}$
$\{r = n!\}$

Avec l'expérience, uniquement pour certaines règles :

- P' et Q' de la règle (*consequence*)
- Invariants de boucle

Annotation du code par des assertions

A priori, entre chaque instruction

$\{n \geq 0\}$
$i := 0;$
$\{a_1\}$
$r := 1;$
$\{a_2\}$
while $i < n$ do
  $\{a_3\}$
  $i := i + 1;$
  $\{a_4\}$
  $r := r * i$
  $\{a_5\}$
od
$\{a_6\}$
$\{r = n!\}$

Avec l'expérience, uniquement pour certaines règles :

- P' et Q' de la règle (*consequence*)
- Invariants de boucle

$\{n \geq 0\}$
$i := 0;$
$r := 1;$
$\{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{20}]}\}$
while $i < n$ do
  $\{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{21}]}\}$
  $i := i + 1;$
  $r := r * i$
  $\{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{22}]}\}$
od
$\{\ldots\ldots\ldots\ldots\ldots\ldots^{[R_{23}]}\}$
$\{r = n!\}$

Assertions principales

Vers la notion de conditions de vérification

Avec ces annotations, d'après les règles (*sequence*) et (*while*), il suffit de démontrer deux triplets de Hoare et une implication :

$\{n \geq 0\}$
$i := 0;$
$r := 1;$
$\{i \leq n \wedge r = i!\}$ — $\{n \geq 0\}\ i := 0;\ r := 1\ \{i \leq n \wedge r = i!\}$
while $i < n$ do
  $\{i < n \wedge i \leq n \wedge r = i!\}$
  $i := i + 1;$
  $r := r * i$ — $\{i < n \wedge i \leq n \wedge r = i!\}$
  $\{i \leq n \wedge r = i!\}$ — $i := i + 1;\ r := r * i\ \{i \leq n \wedge r = i!\}$
od
$\{i \not< n \wedge i \leq n \wedge r = i!\}$
$\{r = n!\}$ — $i \not< n \wedge i \leq n \wedge r = i! \Rightarrow r = n!$

- Modulaire : preuves séparées, sous-arbres plus petits
- Annotations : commentaires ou instructions ?